

Rezolvare detaliată - Examen PAOJ

Programare Avansată pe Obiecte în Java
Sesiunea iunie 2025, Varianta 1

Rezumat răspunsuri grilă:

Ex. 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5
c) 3	d) Matei Ion	b)	b) A B B	a)

1. Subiectul I - Întrebări grilă

1.1. Exercițiul 1 - HashSet, hashCode() și equals()

Programul definește o clasă Automobil cu:

```
public int hashCode() { return marca.length() / 2; }  
public boolean equals(Object obj) {  
    return marca.charAt(0) != ((Automobil)obj).marca.charAt(0);  
}
```

Se adaugă în HashSet obiectele cu mărcile:

Marcă	marca.length() / 2	Prima literă
Dacia	5 / 2 = 2	D
Bugatti	7 / 2 = 3	B
Ford	4 / 2 = 2	F
Borgward	8 / 2 = 4	B
Fiat	4 / 2 = 2	F
Delage	6 / 2 = 3	D

Un HashSet folosește mai întâi hashCode() pentru a alege bucket-ul, iar equals() este verificat doar între obiecte care ajung în același bucket.

Adăugările se analizează astfel:

1. Dacia: bucket cu hash 2 este gol, deci se adaugă.
2. Bugatti: hash 3, bucket gol, deci se adaugă.
3. Ford: hash 2, se compară cu Dacia. Ford.equals(Dacia) întoarce true, deoarece 'F' != 'D', deci Ford este considerat deja existent și nu se adaugă.
4. Borgward: hash 4, bucket gol, deci se adaugă.
5. Fiat: hash 2, se compară cu Dacia. 'F' != 'D', deci equals() întoarce true; nu se adaugă.
6. Delage: hash 3, se compară cu Bugatti. 'D' != 'B', deci equals() întoarce true; nu se adaugă.

În multime rămân 3 obiecte: Dacia, Bugatti, Borgward.

Răspuns: c) 3

Capcană: Metoda equals() este greșită conceptual: două obiecte sunt considerate egale dacă au prima literă diferită. În plus, un obiect nici măcar nu este egal cu alt obiect care are aceeași primă literă, ceea ce încalcă ideea normală de egalitate. Pentru examen, însă, urmărim exact codul dat.

1.2. Exercițiul 2 - Referințe, obiecte și supraîncărcare de metode

Clasa are două metode cu același nume, dar cu parametri diferiți:

```
public void modificaSir(String sir) { this.sir = sir; }  
public void modificaSir(Sir sir) { sir = new Sir("Mihai"); }
```

Analizăm pas cu pas:

```
Sir s = new Sir("Alex"); // s.sir = "Alex"  
Sir t = new Sir("Ion");  // t.sir = "Ion"  
  
s.modificaSir("Matei");
```

Se apelează metoda cu parametru String, deci:

```
this.sir = sir;
```

Așadar s.sir devine "Matei".
Apoi:

```
t.modificaSir(new Sir("Dan"));
```

Argumentul este de tip Sir, deci se apelează metoda:

```
public void modificaSir(Sir sir) { sir = new Sir("Mihai"); }
```

Aici sir este doar o variabilă locală, adică o copie a referinței primite ca parametru. Reatribuirea ei către new Sir("Mihai") nu modifică obiectul t. Deci t.sir rămâne "Ion".

Apoi:

```
s.modificaSir(t);
```

Se apelează tot metoda cu parametru Sir. Din nou, se modifică doar parametrul local sir, nu câmpul s.sir. Prin urmare s.sir rămâne "Matei".

La final:

```
System.out.println(s.getSir() + " " + t.getSir());
```

se afișează:

```
Matei Ion
```

Răspuns: d) Matei Ion

Capcană: În Java, referințele sunt transmise prin valoare. Dacă în metodă faci sir = new Sir("Mihai"), schimbi doar copia locală a referinței, nu obiectul primit și nu referința din apelant.

1.3. Exercițiul 3 - Scanner, excepții și finally

Codul relevant este:

```
Scanner fin = new Scanner(new File("exemplu.txt"));
int s = 0;
try {
    while(fin.hasNext()) {
        int x = fin.nextInt();
        s = s + x;
    }
}
catch(InputMismatchException e) {
    System.out.println("Numar incorect!");
}
finally {
    System.out.println(s);
}
```

Important:

- `hasNext()` verifică dacă mai există un token, nu dacă token-ul este neapărat întreg.
- `nextInt()` încearcă să citească un întreg. Dacă următorul token nu este întreg, se aruncă `InputMismatchException`.
- Când apare excepția, execuția blocului `try` se oprește imediat. Nu se revine în `while` ca să citească următoarele valori.
- Blocul `catch` se execută o singură dată pentru prima valoare incorectă întâlnită.
- Blocul `finally` se execută la final și afișează suma numerelor citite corect înainte de primul token incorect.

Dacă în fișier există cel puțin două numere scrise incorect, acest lucru nu contează, deoarece programul se oprește din citirea fișierului la primul token incorect.

Răspuns: b) Va afișa o singură dată mesajul Numar incorect! și suma numerelor aflate înaintea primului număr scris incorect, după care programul se termină normal.

1.4. Exercițiul 4 - Polimorfism și redefinirea metodelor

Clasele sunt:

```
class A { public void print() { System.out.print("A "); } }
class B extends A { }
class C extends B { public void print() { System.out.print("B "); } }
```

În main:

```
A p = new B();
B q = new C();
A r = new C();

p.print();
q.print();
r.print();
```

Analiză:

- p are tip declarat A, dar tip real B. Clasa B nu redefineste print(), deci se folosește metoda moștenită din A. Se afișează A.
- q are tip declarat B, dar tip real C. Clasa C redefineste print(), deci se afișează B.
- r are tip declarat A, dar tip real C. Pentru metode de instanță, implementarea se alege după tipul real al obiectului. Se afișează B.

Output-ul este:

```
A B B
```

Răspuns: b) A B B

Capcană: La metodele nestatice redefinite contează tipul real al obiectului, nu tipul declarat al referinței.

1.5. Exercițiul 5 - Eliminarea perechilor ()

Codul este:

```
int n = s.length();
int p = s.indexOf("(");
while (p != -1) {
    s = s.substring(0, p) + s.substring(p + 2);
    n -= 2;
    p = s.indexOf("(");
}
System.out.println(n);
```

Programul caută prima apariție a subșirului "(" și o elimină. După fiecare eliminare, lungimea memorată în n scade cu 2. Procesul continuă până când în șir nu mai există nicio pereche adiacentă "(".

Interpretare:

- Orice pereche "(" eliminată corespunde unei perechi de paranteze care se închide corect.
- După eliminarea repetată a tuturor perechilor corecte, rămân doar parantezele care nu pot fi potrivite.
- Numărul rămas, adică n, este numărul minim de paranteze care trebuie inserate pentru ca șirul să devină corect parantezat.

Exemplu:

```
s = "())("
Eliminam "(" -> ramane ")("
Nu mai exista "("
n = 2.
```

Trebuie inserate două paranteze: una '(' înainte de prima ')' și una ')' după ultima '('.

Răspuns: a) Numărul minim de paranteze închise sau deschise care trebuie inserate în șirul s astfel încât toate parantezele din șir să se închidă corect.

2. Subiectul II - Clasa Imobil și prelucrări cu stream-uri

Cerința cere o clasă Imobil cu membrii tipImobil, nrCamere, suprafata, localitate, plus constructori, getteri/setteri, toString(), equals() și hashCode(). Apoi trebuie create cel puțin 3 obiecte și rezolvate cerințele cu stream-uri și lambda expresii.

2.1. Cod complet

```
import java.util.*;
import java.util.stream.Collectors;

class Imobil {
    private String tipImobil;
    private int nrCamere;
    private double suprafata;
    private String localitate;

    public Imobil() {
    }

    public Imobil(String tipImobil, int nrCamere, double suprafata, String
    ↪ localitate) {
        this.tipImobil = tipImobil;
        this.nrCamere = nrCamere;
        this.suprafata = suprafata;
        this.localitate = localitate;
    }

    public String getTipImobil() {
        return tipImobil;
    }

    public void setTipImobil(String tipImobil) {
        this.tipImobil = tipImobil;
    }

    public int getNrCamere() {
        return nrCamere;
    }

    public void setNrCamere(int nrCamere) {
        this.nrCamere = nrCamere;
    }

    public double getSuprafata() {
        return suprafata;
    }

    public void setSuprafata(double suprafata) {
        this.suprafata = suprafata;
    }

    public String getLocalitate() {
        return localitate;
    }

    public void setLocalitate(String localitate) {
        this.localitate = localitate;
    }
}
```

```

@Override
public String toString() {
    return "Imobil{" +
        "tipImobil='" + tipImobil + '\'' +
        ", nrCamere=" + nrCamere +
        ", suprafata=" + suprafata +
        ", localitate='" + localitate + '\'' +
        '}';
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Imobil)) return false;
    Imobil imobil = (Imobil) o;
    return nrCamere == imobil.nrCamere &&
        Double.compare(imobil.suprafata, suprafata) == 0 &&
        Objects.equals(tipImobil, imobil.tipImobil) &&
        Objects.equals(localitate, imobil.localitate);
}

@Override
public int hashCode() {
    return Objects.hash(tipImobil, nrCamere, suprafata, localitate);
}
}

public class TestImobileStreamuri {
    public static void main(String[] args) {
        List<Imobil> imobile = new ArrayList<>();
        imobile.add(new Imobil("apartament", 2, 60.5, "Bucuresti"));
        imobile.add(new Imobil("garsoniera", 1, 38.0, "Bucuresti"));
        imobile.add(new Imobil("casa", 4, 120.0, "Brasov"));
        imobile.add(new Imobil("apartament", 3, 82.0, "Bucuresti"));
        imobile.add(new Imobil("apartament", 2, 70.0, "Cluj"));

        // a) Imobilele din Bucuresti, sortate crescator dupa numarul de
        ↪ camere
        System.out.println("a) Imobile din Bucuresti sortate dupa
        ↪ nrCamere:");
        imobile.stream()
            .filter(i -> i.getLocalitate().equals("Bucuresti"))
            .sorted((i1, i2) -> Integer.compare(i1.getNrCamere(),
            ↪ i2.getNrCamere()))
            .forEach(i -> System.out.println(i));

        // b) Localitatile distincte
        System.out.println("\nb) Localitati distincte:");
        imobile.stream()
            .map(i -> i.getLocalitate())
            .distinct()
            .forEach(loc -> System.out.println(loc));

        // c) Lista imobilelor cu suprafata intre 50 si 100 m^2
        List<Imobil> imobileSuprafata = imobile.stream()
            .filter(i -> i.getSuprafata() >= 50 && i.getSuprafata() <=
            ↪ 100)
            .collect(Collectors.toList());

        System.out.println("\nc) Imobile cu suprafata intre 50 si 100 m^2:");
    }
}

```

```

        imobileSuprafata.forEach(i -> System.out.println(i));

        // d) Numarul de imobile cu doua camere din fiecare localitate
        System.out.println("\nd) Numar imobile cu doua camere pe
        ↳ localitate:");
        Map<String, Long> nrDouaCamerePeLocalitate = imobile.stream()
                .filter(i -> i.getNrCamere() == 2)
                .collect(Collectors.groupingBy(i -> i.getLocalitate(),
                ↳ Collectors.counting()));

        nrDouaCamerePeLocalitate.forEach((loc, nr) ->
                System.out.println(loc + " -> " + nr));
    }
}

```

2.2. Explicația cerințelor

- La punctul a), filter păstrează doar imobilele din București, iar sorted le ordonează crescător după nrCamere.
- La punctul b), map transformă fiecare imobil în localitatea lui, iar distinct elimină duplicatele.
- La punctul c), filter selectează imobilele cu suprafata în intervalul [50,100], iar collect(Collectors.toList()) creează lista cerută.
- La punctul d), filtrăm imobilele cu exact două camere, apoi grupăm după localitate cu Collectors.groupingBy. Pentru fiecare grup, Collectors.counting() numără elementele.

3. Subiectul III - Fișiere text și fir de executare dedicat

Datele sunt în fișiere text. Fiecare linie are forma:

```
tipImobil,nrCamere,suprafata,localitate
```

Trebuie calculat numărul total de imobile dintr-o localitate, pe baza fișierelor imobile_1.txt și imobile_2.txt, folosind câte un fir de executare pentru citirea fiecărui fișier.

3.1. Ideea rezolvării

- Citim de la tastatură localitatea căutată.
- Pornim două fire de executare: unul pentru imobile_1.txt, unul pentru imobile_2.txt.
- Fiecare fir citește propriul fișier și își calculează propriul contor.
- În main, apelăm join() pentru ambele fire, ca să așteptăm terminarea lor.
- Totalul este suma celor două contoare.

3.2. Cod complet

```

import java.io.*;
import java.util.Scanner;

```

```

class NumaratorImobile extends Thread {
    private final String numeFisier;
    private final String localitateCautata;
    private int numarImobile;

    public NumaratorImobile(String numeFisier, String localitateCautata) {
        this.numeFisier = numeFisier;
        this.localitateCautata = localitateCautata;
        this.numarImobile = 0;
    }

    @Override
    public void run() {
        try (BufferedReader br = new BufferedReader(new
            ↪ FileReader(numeFisier))) {
            String linie;

            while ((linie = br.readLine()) != null) {
                String[] campuri = linie.split(",");

                // campuri[0] = tipImobil
                // campuri[1] = nrCamere
                // campuri[2] = suprafata
                // campuri[3] = localitate
                String localitate = campuri[3].trim();

                if (localitate.equalsIgnoreCase(localitateCautata.trim())) {
                    numarImobile++;
                }
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    public int getNumarImobile() {
        return numarImobile;
    }
}

public class TestNumarImobile {
    public static void main(String[] args) throws InterruptedException {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Localitatea cautata: ");
        String localitate = scanner.nextLine();

        NumaratorImobile fir1 = new NumaratorImobile("imobile_1.txt",
            ↪ localitate);
        NumaratorImobile fir2 = new NumaratorImobile("imobile_2.txt",
            ↪ localitate);

        fir1.start();
        fir2.start();

        fir1.join();
        fir2.join();

        int total = fir1.getNumarImobile() + fir2.getNumarImobile();
    }
}

```



```

        System.out.println("Numar total de imobile in " + localitate + ": " +
        ↪ total);
    }
}

```

3.3. Explicație importantă pentru examen

- `start()` pornește un fir nou de executare. Dacă am apela direct `run()`, codul s-ar executa secvențial în firul principal.
- `join()` obligă firul principal să aștepte terminarea firelor `fir1` și `fir2`. Fără `join()`, totalul ar putea fi calculat înainte ca firele să termine citirea fișierelor.
- Fiecare fir are propriul câmp `numarImobile`, deci nu apare problemă de acces concurent asupra aceluiași contor.
- În `run()` am folosit `try-with-resources`, astfel încât fișierul se închide automat.

4. Subiectul IV - Clasă imutabilă care conține obiecte mutabile

Avem o clasă mutabilă `Adresa`, iar clasa `Imobil` trebuie să fie imutabilă. Clasa `Imobil` conține:

- o referință către un obiect `Adresa`;
- un număr natural `numarLocatari`;
- o listă de `StringBuilder`, unde fiecare `StringBuilder` conține numele unui locatar.

Problema este că atât `Adresa`, cât și `StringBuilder`, cât și `List` sunt mutabile. Deci nu este suficient să folosim `final`. Trebuie să facem copii defensive.

4.1. Reguli pentru imutabilitate

O clasă imutabilă trebuie să respecte următoarele reguli:

1. clasa să fie `final`, ca să nu poată fi extinsă de o subclasă mutabilă;
2. câmpurile să fie `private final`;
3. să nu existe setteri;
4. în constructor să se facă deep copy pentru obiectele mutabile primite ca parametri;
5. în getteri să nu se returneze referințele interne mutabile, ci copii.

4.2. Cod complet

```

import java.util.*;

class Adresa {
    private String strada;
    private int numar;
    private String bloc;
    private String scara;
    private int etaj;
    private int apartament;
}

```

```

private String judet;
private String localitate;
private String codPostal;

public Adresa(String strada, int numar, String bloc, String scara,
               int etaj, int apartament, String judet,
               String localitate, String codPostal) {
    this.strada = strada;
    this.numar = numar;
    this.bloc = bloc;
    this.scara = scara;
    this.etaj = etaj;
    this.apartament = apartament;
    this.judet = judet;
    this.localitate = localitate;
    this.codPostal = codPostal;
}

// Constructor de copiere, necesar pentru copii defensive.
public Adresa(Adresa alta) {
    this.strada = alta.strada;
    this.numar = alta.numar;
    this.bloc = alta.bloc;
    this.scara = alta.scara;
    this.etaj = alta.etaj;
    this.apartament = alta.apartament;
    this.judet = alta.judet;
    this.localitate = alta.localitate;
    this.codPostal = alta.codPostal;
}

// Setteri, fiindca Adresa este mutabila.
public void setStrada(String strada) {
    this.strada = strada;
}

public void setLocalitate(String localitate) {
    this.localitate = localitate;
}

@Override
public String toString() {
    return strada + " nr. " + numar + ", bloc " + bloc +
           ", scara " + scara + ", etaj " + etaj +
           ", ap. " + apartament + ", " + localitate +
           ", " + judet + ", " + codPostal;
}
}

final class Imobil {
    private final Adresa adresa;
    private final int numarLocatari;
    private final List<StringBuilder> locatari;

    public Imobil(Adresa adresa, int numarLocatari, List<StringBuilder>
        ↪ locatari) {
        // Copie defensivă pentru Adresa, deoarece Adresa este mutabilă.
        this.adresa = new Adresa(adresa);

        this.numarLocatari = numarLocatari;
    }
}

```

```

        // Copie defensivă pentru lista și pentru fiecare StringBuilder din
        ↪ lista.
        List<StringBuilder> copieLocatari = new ArrayList<>();
        for (StringBuilder locatar : locatari) {
            copieLocatari.add(new StringBuilder(locatar.toString()));
        }

        // Lista internă nu poate fi modificată structural.
        this.locatari = Collections.unmodifiableList(copieLocatari);
    }

    public Adresa getAdresa() {
        // Nu returnăm referința internă, ci o copie.
        return new Adresa(adresa);
    }

    public int getNumarLocatari() {
        return numarLocatari;
    }

    public List<StringBuilder> getLocatari() {
        // Returnăm o listă nouă, cu StringBuilder-uri noi.
        List<StringBuilder> copie = new ArrayList<>();
        for (StringBuilder locatar : locatari) {
            copie.add(new StringBuilder(locatar.toString()));
        }
        return Collections.unmodifiableList(copie);
    }

    @Override
    public String toString() {
        return "Imobil{" +
            "adresa=" + adresa +
            ", numarLocatari=" + numarLocatari +
            ", locatari=" + locatari +
            '}';
    }
}

```

4.3. De ce este corectă soluția

Să presupunem că am scrie greșit constructorul astfel:

```

this.adresa = adresa;
this.locatari = locatari;

```

Atunci codul din exterior ar putea modifica obiectul `Imobil` după construire:

```

Adresa a = new Adresa(...);
List<StringBuilder> lista = new ArrayList<>();
lista.add(new StringBuilder("Ion"));

Imobil imobil = new Imobil(a, 1, lista);

a.setLocalitate("Alta localitate"); // ar modifica adresa internă, dacă nu
↪ copiem
lista.get(0).append(" Popescu");    // ar modifica locatarul intern, dacă nu
↪ copiem

```

Soluția corectă face următoarele:

- în constructor, `new Adresa(adresa)` rupe legătura cu obiectul `Adresa` primit din exterior;
- în constructor, pentru fiecare `StringBuilder` se creează un alt `StringBuilder`;
- în getterul `getAdresa()`, se returnează o copie, nu câmpul intern;
- în getterul `getLocatari()`, se returnează o listă nouă cu elemente noi;
- `Collections.unmodifiableList` împiedică modificarea structurală a listei returnate.

Capcană: `final` aplicat pe o referință înseamnă că referința nu mai poate indica alt obiect, dar obiectul indicat poate fi în continuare mutabil. De aceea sunt necesare copii defensive.

5. Concluzii rapide pentru examen

- La `HashSet`, verifică întâi `hashCode()`, apoi `equals()` doar în același bucket.
- La referințe, Java transmite parametrii prin valoare. Reatribuirea parametrului nu modifică referința din apelant.
- La excepții, după ce apare o excepție în `try`, execuția blocului `try` se oprește. `finally` se execută oricum.
- La polimorfism, metodele nestatice redefinite se aleg după tipul real al obiectului.
- La stream-uri, reține schema: `stream()` → operații intermediare → operație terminală.
- La fire, folosește `start()` pentru fir nou și `join()` dacă firul principal trebuie să aștepte rezultatul.
- La clase imutabile cu obiecte mutabile în interior, folosește câmpuri private `final`, fără setteri și copii defensive în constructori și getteri.